DASH 实验报告：BBA-0 算法实现与分析

严子竣

2026 年 4 月 4 日

1 搭建流程

1. 获取 dash.js，安装 node.js 和 npm 并配置环境变量等流程，与所提供的文档一致，此处不再赘述。

2. 使用虚拟机（Ubuntu 22.04）配置服务器：安装 server 版，搭建结果如图1和图2所示：

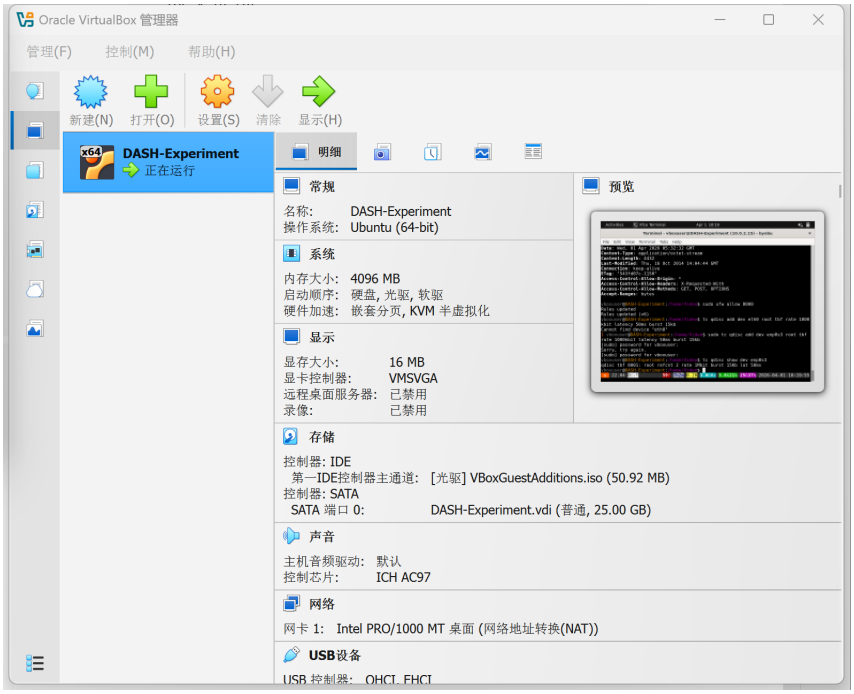

图 1: 虚拟机

图 2: 端口转发规则

3. 安装 nginx

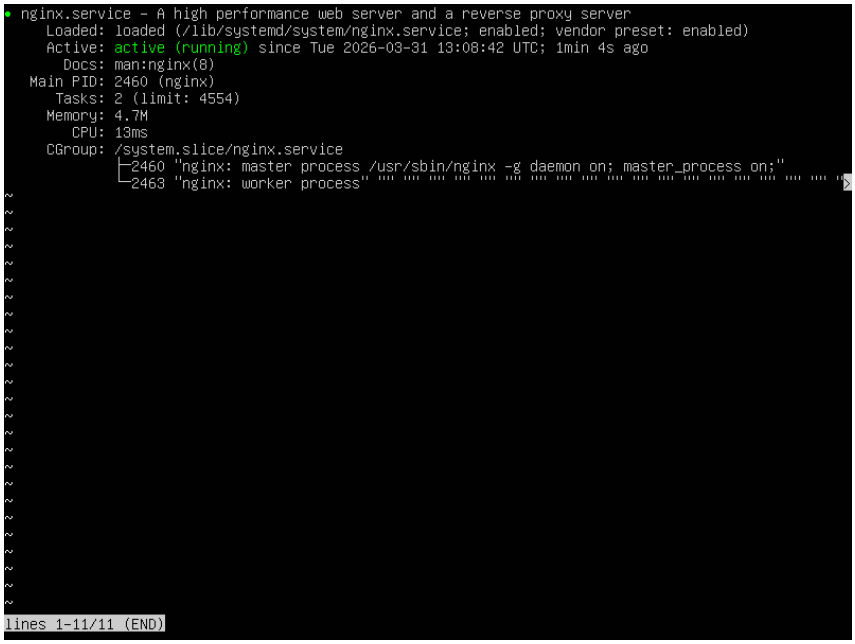

图 3: nginx 安装结果

安装 nginx 后，修改 Nginx 配置文件 `/etc/nginx/nginx.conf`，添加如下 server 块：

```
server {
    listen 0.0.0.0:8000;
    server_name 127.0.0.1;
    add_header Access-Control-Allow-Origin *;
    add_header Access-Control-Allow-Headers X-Requested-With;
    add_header Access-Control-Allow-Methods GET,POST,OPTIONS;
    location / {
```

```
        root /home/video;
        autoindex on;
    }
}
```

重启 Nginx：`sudo systemctl restart nginx`。

通过浏览器访问 `http://10.0.2.15:8000/` 能看到视频文件列表，证明服务器配置成功。

4. 获取视频文件：进入目录/home/Video，使用 wget 递归下载 2sec 版本的视频文件：

```
cd /home/Video
wget -r -np -nH --cut-dirs=4 -R "index.html*" http://ftp.itec.aau.at/datasets/
    DASHDataset2014/BigBuckBunny/2sec/
```

再使用 scp 将视频上传至服务器（未及时保存截图，找不到了）。

在 dash.js 中输入：

`http://127.0.0.1:8000/BigBuckBunny/2sec/BigBuckBunny_2s_simple_2014_05_09.mpd`

能够正常播放，说明上传成功。

5. 使用 tc 限制服务器的网络带宽

```
sudo tc qdisc add dev enp0s3 root tbf rate 1000kbit latency 50ms burst 15kb
```

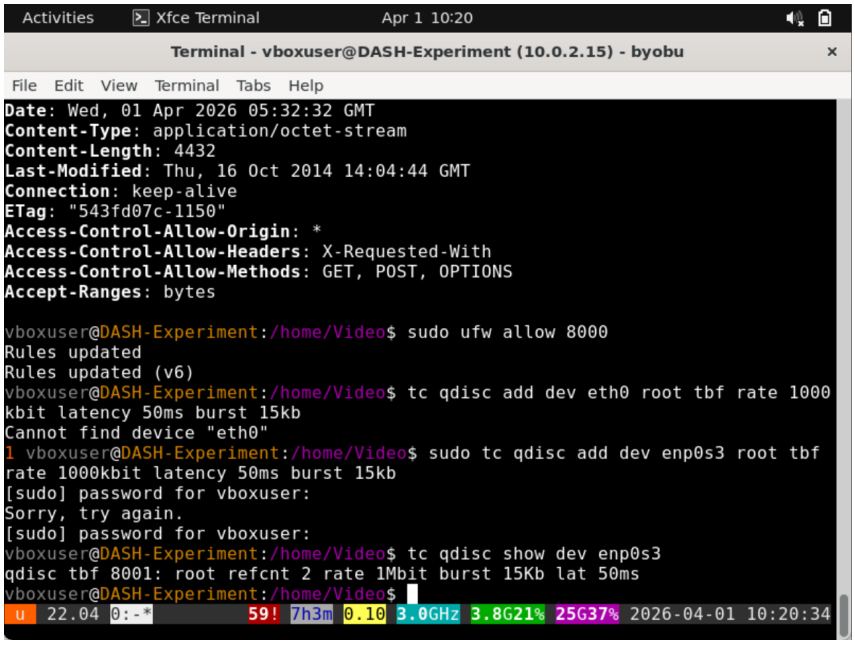

图 4: 使用 tc 限制带宽

2 实现的 ABR 算法：BBA-0

BBA-0 完全基于缓冲区长度（秒）决策码率，定义低阈值 $L = 10$ 秒、高阈值 $H = 30$ 秒：

- 缓冲区 $< L$：选择最低码率。
- 缓冲区 $> H$：选择最高码率。

- 缓冲区在 $[L, H]$：线性插值

$$\text{index} = \left\lfloor \frac{B - L}{H - L} \times (N - 1) \right\rfloor$$

其中 N 为码率等级数。

在 dash.js 中新建 `BBARule.js`，实现 `getMaxIndex` 方法，注册到 `AbrController`，并在 HTML 界面添加“BBA-0”选项。

代码如下：

Listing 1: BBARule.js

```js
import FactoryMaker from '../../../core/FactoryMaker.js';

function BBARule() {
    const lowThreshold = 10;
    const highThreshold = 30;

    const instance = {
        getMaxIndex: function (rulesContext) {
            const abrController = rulesContext.getAbrController();
            const mediaInfo = rulesContext.getMediaInfo();
            const bufferLevel = rulesContext.getBufferState().getBufferLevel();
            const bitrateList = abrController.getBitrateList(mediaInfo);

            if (!bitrateList || bitrateList.length === 0) return -1;
            if (bufferLevel === undefined) return bitrateList.length - 1;

            let targetBitrateIndex;

            if (bufferLevel <= lowThreshold) {
                targetBitrateIndex = 0; // 最低码率
            } else if (bufferLevel >= highThreshold) {
                targetBitrateIndex = bitrateList.length - 1; // 最高码率
            } else {
                // 线性插值
                const ratio = (bufferLevel - lowThreshold) / (highThreshold -
                    lowThreshold);
                const maxIndex = bitrateList.length - 1;
                targetBitrateIndex = Math.floor(ratio * maxIndex);
            }

            // 保证索引有效
            targetBitrateIndex = Math.min(Math.max(targetBitrateIndex, 0),
                bitrateList.length - 1);
            return targetBitrateIndex;
        }
    };

    return instance;
}

```

```
BBARule.__dashjs_factory_name = 'BBARule';
export default FactoryMaker.getClassFactory(BBARule);
```

3 生效结果与分析

使用 tc 限制带宽为 2Mbps，播放时观察客户端下方监控图（图5）。当缓冲区低于 10 秒时，码率立即降为最低档（约 0.5 Mbps）；缓冲区上升至 10-30 秒区间时，码率随缓冲区线性增加；超过 30 秒后稳定在最高码率。带宽恢复后，缓冲区重新积累，码率平缓回升。整个过程无卡顿，验证了 BBA-0 的鲁棒性。

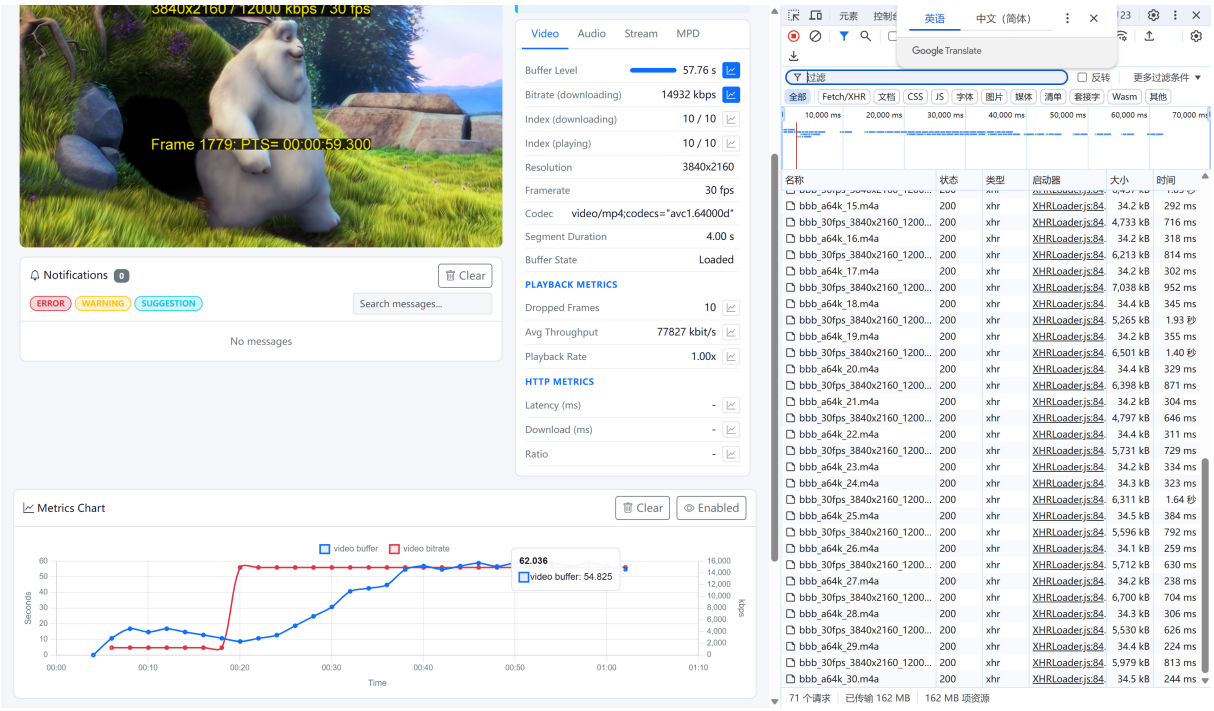

图 5: 客户端码率与缓冲区监控图

4 学习总结

- **DASH**：将视频切分为分片并提供 MPD 描述文件，客户端自主请求合适码率的分片，实现了服务器与客户端的解耦。
- **BBA**：仅根据当前缓冲区占用量（buffer occupancy）来决定请求的码率，完全不依赖网络吞吐量预测，从而避免吞吐量预测不准带来的问题。